Ágiles 2026-2 · Ana Sofía Arango Yanza · Juan José Arias Chacua · 02/11/2026 · v7

# Ficha t21 · Fusionar cambios de dos dispositivos

Con agente, individual · tamaño L (grande · 45–70 min)

## 1 Antes de empezar

| |
|---|
| Fecha · sesión · ciclo (1 o 2) |
| Quién conduce (solo, con el agente) |
| **M5 · Pronóstico** en minutos (después de leer) |

En tu repo corre `git pull`. Lee la sección **t21** de `TAREAS.md`, máximo 2 minutos, sin programar ni abrir el código, y anota el pronóstico.

## 2 Durante (el tiempo corre)

Con el pronóstico anotado, arranca el cronómetro y corre:

```
git commit --allow-empty -m "t21: inicio"
```

**En la copia** (`notitas-agente`), limpia y con conversación nueva: `git checkout -- backend/notitas`, `opencode`, `/new`. Pega este prompt, cambiando el corchete por la sección t21 de `TAREAS.md`:

```
Actúa como navegante de pair programming (spec Copiloto v1). Tarea: t21.
Este es el enunciado de la tarea:
[PEGAR AQUÍ la sección t21 de TAREAS.md: desde la línea "## t21 ·" hasta antes del siguiente "## t"]

Archivos: @backend/notitas/t21.py @backend/tests/visibles/test_t21.py
No leas otros archivos ni pidas las pruebas ocultas. No hagas commit. No edites pruebas. No inventes requisitos que no estén en el enunciado.
Responde en markdown con estas 4 partes:
1. Plan de 3 a 6 pasos.
2. Cambio propuesto, solo en backend/notitas/t21.py.
3. Supuestos que hiciste.
4. Casos borde que crees que faltan (como texto, no como pruebas).
```

Cada propuesta del agente es una fila (A = aceptar · C = corregir · R = rechazar):

| # | Qué pidió / qué propuso | A / C / R | Por qué |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 3 | | | |
| 4 | | | |
| 5 | | | |

Prueba en la copia con `python -m pytest -k t21`. Cuando pase todo, **desde tu repo**, copia el archivo:

```
Copy-Item ../notitas-agente/backend/notitas/t21.py backend/notitas/t21.py
```

| |
|---|
| Hora de inicio |
| Pausas (minutos y motivo) |
| Hora de fin (pasan las visibles y ya creen que cumplen los criterios) |
| **M1 · Minutos** (fin – inicio – pausas) |

Al llegar al fin, para el cronómetro y guarda:

```
git add backend/notitas/t21.py
git commit -m "t21: terminado"
```

## 3 Después de "terminado"

Corre las ocultas en **tu repo**, nunca en la copia: `python -m pytest backend/tests/ocultas -k t21`. Si corriges, no le muestres al agente las pruebas ocultas.

| | |
|---|---|
| **M2 · Ocultas que fallan** | de 3 |
| **M3 · Minutos corrigiendo** (0 si no falló ninguna) | |
| **M4 · Codewalk autor** (0–5) | |
| **M4 · Codewalk compañero** (0–5) | |
| ¿Abrieron las ocultas antes? Si sí: **[C]** y usen una de repuesto | ☐ No ☐ Sí |
| Observaciones y nombre del video | |

**Si falló alguna:** anoten M2 primero. Luego arranquen otro cronómetro y corrijan hasta que pasen todas, sin límite de tiempo, con `python -m pytest -k t21` las veces que necesiten. Ese tiempo es M3; M1 y M2 no cambian. Después, el codewalk. Al final de la tarea, siempre suban:

```
git add backend/notitas/t21.py
git commit -m "t21: retrabajo"     (solo si corrigieron)
git push
```

Codewalk, ya corregida la tarea: Conductor = autor; el otro = compañero (no participó, así que suele saber menos: es normal). Nadie mira el código ni al agente mientras responde. Abran `Guia_Medicion/Codewalk/Por_tarea/02112026_Codewalk_t21_v3.pdf`; cada uno responde por escrito, sin hablar, y luego cada uno califica al otro.
Guarden en `Respuestas/` esta ficha (`t21_llena.pdf` o foto) y las respuestas del codewalk (`t21_codewalk_Nombre.txt`).